



Caio Bonani Carvalho

Implementação de uma Arquitetura Lambda para Processamento de Dados em *Batch* e Tempo Real

São José dos Campos, SP

Caio Bonani Carvalho

Implementação de uma Arquitetura Lambda para Processamento de Dados em *Batch* e Tempo Real

Trabalho de conclusão de curso apresentado ao
Instituto de Ciência e Tecnologia – UNIFESP,
como parte das atividades para obtenção do tí-
tulo de Bacharel em Ciência da Computação.

Universidade Federal de São Paulo – UNIFESP

Instituto de Ciência e Tecnologia

Bacharelado em Ciência da Computação

Orientador: Prof. Dra. Daniela Leal Musa

São José dos Campos, SP

Dezembro de 2025

Este trabalho é dedicado a todos que me ajudaram e estiveram ao meu lado até aqui.

Agradecimentos

Gostaria de expressar minha profunda gratidão a todas as pessoas que contribuíram, direta ou indiretamente, não apenas para a realização deste trabalho, mas também para toda a minha trajetória até aqui.

Agradeço, em primeiro lugar, aos meus pais, pois sem o apoio, incentivo e sacrifício deles meu ingresso e permanência neste curso não teriam sido possíveis.

Estendo meus agradecimentos aos professores que me acompanharam ao longo dessa jornada, em especial aos(as) professores(as) Arlindo Flávio da Conceição, Cláudia Barbosa Ladeira de Campos e Daniela Leal Musa, cujos ensinamentos e orientações foram fundamentais para meu desenvolvimento acadêmico.

Por fim, agradeço aos amigos que fiz durante esses anos. Foi graças ao companheirismo, apoio e convivência com eles que consegui chegar até aqui. Meu muito obrigado ao Eduardo Veríssimo, Enzo Reis, Raphael Damasceno e a todos os outros amigos que fizeram parte dessa caminhada.

*“Computer programming is an art, because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially because it produces objects of beauty.”*
(Donald Knuth)

Resumo

Este trabalho propõe a implementação e avaliação experimental de uma arquitetura Lambda para processamento híbrido de dados massivos. O objetivo é conciliar respostas em tempo real com análises históricas de alta precisão por meio da separação em três camadas: *Batch Layer*, *Speed Layer* e *Serving Layer*. A implementação proposta integrará um pipeline *open-source* composto por Apache Kafka (ingestão), Apache Spark (processamento unificado em *streaming* e *batch*), HDFS e Apache Cassandra (*Serving Layer* para leituras rápidas).

A infraestrutura será orquestrada com Docker e testada em cenários de falha e alta carga. A *Speed Layer* utiliza Spark Structured Streaming para gerar métricas e alertas com latência na ordem de segundos; a *Batch Layer* realiza reprocessamentos completos sobre o HDFS para garantir precisão analítica; a *Serving Layer* consolida as visões para consulta. Os experimentos irão avaliar latência ponta-a-ponta, *throughput* e tempo médio de recuperação (MTTR), além da consistência final entre visões *batch* e *speed*.

Por fim, o trabalho discute as vantagens e limitações do modelo Lambda destacando casos de uso em que a separação entre processamento *batch* e *streaming* ainda se mostra relevante. A análise considera aspectos como custo operacional, complexidade de manutenção, flexibilidade de escalabilidade e compatibilidade com ecossistemas modernos de dados. Os resultados obtidos buscam fornecer diretrizes para profissionais e pesquisadores que pretendem projetar *pipelines* de dados robustos, de baixa latência e alta confiabilidade em ambientes distribuídos.

Palavras-chave: Arquitetura Lambda; Apache Kafka; Apache Spark; HDFS; Apache Cassandra; processamento em tempo real; *batch*.

Abstract

This work proposes the implementation and experimental evaluation of a Lambda architecture for hybrid processing of large-scale data. The goal is to combine low-latency real-time responses with high-precision historical analytics through three layers: Batch, Speed and Serving. The proposed implementation will integrate an open-source stack: Apache Kafka for ingestion, Apache Spark for unified streaming and batch processing, HDFS as the immutable master dataset store, and Apache Cassandra as the serving layer for fast reads.

The infrastructure will be orchestrated using Docker and tested under failure and high-load scenarios. The Speed Layer uses Spark Structured Streaming to produce near-real-time metrics and alerts; the Batch Layer performs full reprocessing on HDFS to ensure analytical accuracy; the Serving Layer consolidates views for querying. Experiments will measure end-to-end latency, throughput and mean time to recovery (MTTR), as well as final consistency between batch and speed views.

Finally, the work discusses the advantages and limitations of the Lambda model, highlighting use cases in which the separation between batch and streaming processing remains relevant. The analysis considers aspects such as operational cost, maintenance complexity, scalability flexibility, and compatibility with modern data ecosystems. The results aim to provide guidelines for professionals and researchers who intend to design robust, low-latency, and highly reliable data pipelines in distributed environments.

Key-words: Lambda Architecture; Apache Kafka; Apache Spark; HDFS; Apache Cassandra; real-time processing; batch.

Lista de ilustrações

Figura 1 – Fluxo de dados projetado	28
Figura 2 – Pipeline <i>Speed Layer</i>	33
Figura 3 – Pipeline da <i>Batch Layer</i>	33
Figura 4 – Monitoramento do sistema.	35
Figura 5 – Arquitetura final do sistema.	37

Lista de tabelas

Tabela 1 – Comparação entre <i>Batch</i> e <i>Speed Layer</i>	34
Tabela 2 – Resumo dos testes de robustez.	36
Tabela 3 – Cronograma de atividades para TCC I e TCC II	39

Lista de abreviaturas e siglas

IoT	Internet das Coisas
RDBMS	Sistemas de Gerenciamento de Banco de Dados Relacionais
HDFS	Hadoop Distributed File System
Kafka	Apache Kafka
Spark	Apache Spark
Cassandra	Apache Cassandra
IDS	Sistema de Detecção de Intrusão
CNNs	Redes Neurais Convolucionais
LSTMs	Long Short-Term Memory
MTTR	Mean Time To Recovery (Tempo Médio de Recuperação)
CAP	Consistency, Availability, Partition Tolerance
TCC	Trabalho de Conclusão de Curso
SUS	Sistema Único de Saúde

Lista de símbolos

Q	Resultado da consulta (<i>Query</i>)
f	Função de processamento de dados
D	Conjunto total de dados (<i>Dataset</i>)
M	<i>Master Dataset</i> (Dados históricos imutáveis)
R	Dados recentes (<i>Speed Layer</i>)
V_b	Visualização da <i>Batch Layer</i>
V_s	Visualização da <i>Speed Layer</i>
$\oplus$	Operador de fusão de visualizações
$MTTR$	Tempo Médio de Recuperação (<i>Mean Time To Recovery</i>)
t_{down}	Tempo total de inatividade do sistema
N_f	Número de falhas observadas

Sumário

1	Introdução	21
1.1	Motivação e Justificativa	21
1.2	Objetivos	22
1.2.1	Objetivo Geral	22
1.2.2	Objetivos Específicos	23
2	Revisão da Literatura	25
3	Fundamentação Teórica	27
3.1	Arquitetura Lambda	27
3.2	Análise e Seleção de Ferramentas	28
3.2.1	Apache Kafka	29
3.2.2	Processamento e Armazenamento: Apache Spark e HDFS	29
3.2.3	<i>Serving Layer</i> – Apache Cassandra	29
4	Metodologia	31
4.1	Cenário de Validação: Dados do SUS	31
4.2	Configuração e Orquestração da Infraestrutura	31
4.2.1	Configuração do Kafka	32
4.2.2	Configuração do HDFS	32
4.3	Implementação da <i>Speed Layer</i>	32
4.4	Implementação da <i>Batch Layer</i>	33
4.5	<i>Serving Layer</i>	33
4.6	Monitoramento e Observabilidade	34
4.7	Testes, Validação e Robustez	35
4.8	Arquitetura Consolidada	36
5	Plano de Trabalho e Cronograma	39
	Referências	41

1 Introdução

1.1 Motivação e Justificativa

O cenário tecnológico contemporâneo é intrinsecamente marcado pela onipresença de dispositivos interconectados e pela geração exponencial de dados, um fenômeno categorizado sob o paradigma de *Big Data*. Este conceito não se restringe apenas ao volume massivo de informações, mas abrange as dimensões de velocidade, variedade, veracidade e valor (os “5Vs”), impondo desafios arquiteturais sem precedentes (AISSI et al., 2023). Organizações de diversos setores, da agricultura de precisão à saúde, enfrentam a necessidade crítica não apenas de armazenar *petabytes* de dados heterogêneos, mas de extrair valor analítico em janelas de tempo que variam de milissegundos a décadas. Nesse contexto, emerge uma dicotomia técnica fundamental: a imperatividade do processamento em tempo real (*stream processing*) para respostas imediatas a eventos críticos, versus a necessidade de processamento em lotes (*batch processing*) para a consolidação de análises históricas profundas, complexas e precisas (MARZ; WARREN, 2015).

Essa tensão torna-se particularmente evidente e crítica em domínios sensíveis à latência, como *Smart Farming* e a Internet das Coisas (IoT). Conforme elucidado por estudos recentes na área de monitoramento agrícola (AISSI et al., 2023; MUNSHI; MOHAMED, 2018), a eficiência operacional moderna depende de redes de sensores que transmitem fluxos contínuos de dados — incluindo umidade do solo, temperatura, imagens de satélite e radiação solar. Para a automação de atuadores, como sistemas de irrigação autônoma, a latência deve ser mínima, exigindo uma arquitetura reativa e de alta disponibilidade. Por outro lado, o planejamento estratégico de safras futuras depende da correlação desses mesmos dados com históricos climáticos de longo prazo, demandando o poder computacional de processamentos massivos em lote que garantam a completude e a integridade da informação (ROUKH et al., 2020).

Entretanto, Sistemas de Gerenciamento de Banco de Dados Relacionais (RDBMS) e arquiteturas monolíticas tradicionais frequentemente se mostram inadequados para conciliar esses requisitos antagônicos. Sob a ótica do Teorema CAP (*Consistency, Availability, Partition Tolerance*), sistemas tradicionais priorizam a consistência forte em detrimento da disponibilidade em cenários de partição de rede, o que é inviável para sistemas distribuídos de larga escala. Ao tentar forçar um único sistema a lidar com ingestão de alta velocidade (escrita) e consultas analíticas complexas (leitura) simultaneamente, surgem gargalos severos de desempenho e bloqueios de recursos (*deadlocks*) (MALUSARE, 2019). Ademais, sistemas tradicionais baseados em atualizações destrutivas — onde o novo dado sobrescreve o antigo — são inerentemente frágeis a erros humanos e falhas lógicas: um algoritmo incorreto pode corromper permanentemente o estado atual do banco de dados, sem possibilidade de recuperação fidedigna do estado

anterior.

A motivação central para este projeto reside, portanto, na superação dessas limitações estruturais através da adoção de uma arquitetura que priorize a imutabilidade, a escalabilidade horizontal e a tolerância a falhas humanas e de *hardware*. A abordagem proposta fundamenta-se nos princípios da Arquitetura Lambda, introduzida por Nathan Marz, que estabelece a premissa de que o dado bruto é “sagrado” e imutável (MARZ; WARREN, 2015). Ao desacoplar as responsabilidades de processamento em camadas distintas, o sistema ganha a capacidade de “recomputação”: caso ocorra um erro na lógica de análise ou surja a necessidade de novos algoritmos, é possível reprocessar todo o *Master Dataset* armazenado na *Batch Layer* para corrigir ou enriquecer as visualizações, sem interromper o fluxo contínuo e de baixa latência da *Speed Layer*. Adicionalmente, a arquitetura oferece robustez contra intrusões e anomalias, permitindo a auditoria de dados brutos para detecção de padrões maliciosos em ambientes IoT (ALGHAMDI; BELLAICHE, 2023).

Dessa forma, a adoção da Arquitetura Lambda justifica-se não apenas pela performance superior em ambientes de alta vazão, mas pela resiliência sistêmica que ela oferece frente à complexidade do *Big Data*. Inspirado em desafios de engenharia de dados modernos (ANDE; SINGH, 2025), este trabalho propõe uma implementação prática que orquestra um ecossistema de ferramentas *open-source* consolidadas e complementares: Apache Kafka para o desacoplamento da ingestão; Apache Spark (*Streaming* e *Batch*) para o processamento unificado; HDFS para o armazenamento persistente e distribuído; e Apache Cassandra para a *Serving Layer*, oferecendo consistência eventual e alta disponibilidade de leitura. Esta escolha tecnológica visa demonstrar que é possível construir uma plataforma escalável, capaz de absorver o crescimento exponencial dos dados sem a necessidade de refatorações custosas de infraestrutura, garantindo sustentabilidade técnica e econômica para aplicações de missão crítica.

1.2 Objetivos

1.2.1 Objetivo Geral

Projetar, implementar e avaliar experimentalmente uma arquitetura de referência baseada no modelo Lambda para o processamento híbrido de dados massivos, orquestrando um ecossistema de *Big Data* composto por Apache Kafka, Apache Spark, HDFS e Apache Cassandra. O foco primordial é validar a capacidade desta arquitetura em garantir baixa latência na *Speed Layer*, alta precisão na *Batch Layer* e consistência eventual na *Serving Layer*, mitigando os compromissos inerentes ao Teorema CAP em cenários de ingestão intensa.

1.2.2 Objetivos Específicos

Para assegurar o cumprimento do objetivo geral e a validação da hipótese de que a arquitetura híbrida supera as limitações de escalabilidade e tolerância a falhas dos modelos tradicionais, foram definidos os seguintes objetivos específicos:

- **Estabelecer um *pipeline* de ingestão desacoplado e resiliente:** Configurar e otimizar o Apache Kafka como *broker* de mensagens distribuído, garantindo que a ingestão de dados (produtores) seja assíncrona e não impactada pela latência de consumo das camadas de processamento, assegurando tolerância a picos de carga (*backpressure*) e persistência temporária dos fluxos de dados.
- **Implementar a *Speed Layer*:** Desenvolver rotinas de processamento de fluxo contínuo utilizando Spark Structured Streaming para a ingestão de micro-lotes, visando a geração de métricas, inferências e alertas operacionais com latência na ordem de segundos, essenciais para tomadas de decisão em tempo real em ambientes dinâmicos.
- **Estruturar a *Batch Layer* Imutável:** Construir um repositório de dados mestre (*Master Dataset*) imutável sobre o sistema de arquivos distribuído HDFS (Hadoop Distributed File System) e desenvolver algoritmos em Spark para o reprocessamento periódico e integral do histórico, garantindo a “verdade absoluta” dos dados e a correção de eventuais desvios gerados pela *Speed Layer*.
- **Integrar a *Serving Layer*:** Modelar esquemas de dados desnormalizados no Apache Cassandra, otimizados para leitura, permitindo a consolidação (*merge*) das visões geradas pelas camadas *batch* e *speed*. O objetivo é fornecer consultas unificadas de alta performance.
- **Realizar análise comparativa de desempenho e resiliência:** Executar testes de carga controlados (*stress testing*) com diferentes volumetrias e velocidades de ingestão para mensurar métricas críticas, tais como: latência de ponta a ponta (*end-to-end latency*), vazão máxima (*throughput*), tempo de recuperação do sistema (MTTR) após falhas induzidas de nós e a integridade dos dados após ciclos de reprocessamento.

2 Revisão da Literatura

A fundamentação teórica para sistemas de processamento de dados em larga escala foi consolidada na obra seminal de (MARZ; WARREN, 2015), que formalizou a Arquitetura Lambda. Os autores argumentam que sistemas tradicionais de banco de dados não conseguem escalar linearmente para atender aos requisitos de *Big Data*, especialmente quando confrontados com o Teorema CAP. A solução proposta baseia-se na premissa da imutabilidade dos dados brutos e na decomposição do sistema em três camadas distintas: a *Batch Layer*, responsável pelo gerenciamento do *master dataset* e pré-computação de visualizações com alta latência e precisão; a *Speed Layer*, que compensa a latência da *Batch Layer* processando dados recentes em tempo real; e a *Serving Layer*, que unifica os resultados para consulta. Esta abordagem permite construir sistemas robustos a falhas humanas e de *hardware*, garantindo consistência eventual.

Expandindo a análise sobre os paradigmas de processamento que compõem essa arquitetura, (BENJELLOUN et al., 2020) realizam um estudo comparativo detalhado entre o processamento baseado em lotes e o processamento baseado em fluxo (*stream*). O estudo destaca que, embora o processamento em lote (como o MapReduce) seja eficiente para grandes volumes de dados históricos, ele falha em cenários que exigem latência próxima de zero. Em contrapartida, o processamento de fluxo lida com dados em movimento, mas enfrenta desafios de gerenciamento de estado e precisão. Os autores concluem que a integração híbrida desses modelos é essencial para extrair valor de dados heterogêneos, servindo como justificativa técnica para a adoção de arquiteturas como a Lambda em ambientes corporativos modernos.

No tocante à implementação prática e aos desafios de engenharia de software, (MALUSARE, 2019) investiga a aplicação da Arquitetura Lambda utilizando tecnologias *open-source* modernas, especificamente Apache Kafka, Apache Spark e bancos NoSQL. O trabalho demonstra que sistemas de gerenciamento de banco de dados relacionais (RDBMS) tornam-se gargalos operacionais quando submetidos a taxas de ingestão massiva típicas de aplicações web e IoT. O autor valida experimentalmente que a separação das camadas permite escalar horizontalmente os componentes de ingestão e processamento de forma independente, garantindo que a análise em tempo real não seja impactada por operações intensivas de escrita ou reproprocessamento histórico. Adicionalmente, revisões recentes sobre *data warehousing* em tempo real (QU; WANG, 2024) reforçam a necessidade de arquiteturas que suportem ingestão contínua sem degradação de performance.

Aplicando esses conceitos ao domínio da agricultura de precisão, (ROUKH et al., 2020) propõem uma arquitetura de processamento de *Big Data* denominada WALLESMART, projetada para superar a fragmentação de dados em fazendas inteligentes. O estudo identifica que

a maioria das soluções existentes são proprietárias ou não padronizadas, dificultando a interoperabilidade. A arquitetura proposta utiliza componentes da Arquitetura Lambda para ingerir dados de sensores IoT e drones, processando-os para fornecer monitoramento em tempo real das culturas. O trabalho enfatiza a necessidade de uma *Batch Layer* robusta para analisar tendências sazonais, permitindo aos agricultores maximizar a produtividade com base em históricos climáticos e de solo.

Aprofundando-se na convergência entre *Data Lakes* e arquiteturas de fluxo para o setor agrícola, (AISSI et al., 2023) apresentam uma plataforma escalável que integra a Arquitetura Lambda com o conceito de *Data Lake*. A pesquisa aborda a complexidade dos dados gerados por dispositivos IoT na agricultura moderna, que variam em volume, velocidade e variedade. A solução proposta implementa um *pipeline* onde o Kafka atua como *broker* central, alimentando simultaneamente um Spark Streaming para alertas imediatos (como falhas na irrigação) e um armazenamento HDFS para análises profundas. Os resultados indicam que essa abordagem híbrida resolve o problema da subutilização de dados gerados no campo, fornecendo uma infraestrutura resiliente para suporte à decisão.

Em um domínio correlato de infraestrutura crítica, (MUNSHI; MOHAMED, 2018) desenvolvem um ecossistema para *Smart Grids* (Redes Inteligentes) fundamentado na Arquitetura Lambda. O trabalho aborda o desafio de gerenciar dados heterogêneos provenientes de medidores inteligentes (*smart meters*), que incluem não apenas leituras numéricas, mas também imagens e vídeos de monitoramento da rede elétrica. Os autores demonstram, através de uma implementação em nuvem, que a arquitetura é capaz de realizar operações paralelas de processamento em lote e em tempo real sobre dados distribuídos, permitindo a detecção instantânea de anomalias na rede enquanto mantém um histórico auditável para faturamento e planejamento energético.

Por fim, abordando a segurança cibernética em ambientes de IoT, (ALGHAMDI; BELLAICHE, 2023) propõem um Sistema de Detecção de Intrusão (IDS) baseado em aprendizado profundo (*Deep Learning*) estruturado sobre a Arquitetura Lambda. O estudo inova ao utilizar a *Batch Layer* para o treinamento pesado de modelos complexos (como CNNs e LSTMs) utilizando grandes volumes de tráfego histórico, enquanto a *Speed Layer* utiliza esses modelos treinados para classificar tráfego e detectar ataques em tempo real. Esta aplicação demonstra a versatilidade da arquitetura não apenas para análise de dados de negócios, mas como um *framework* fundamental para a segurança de redes distribuídas, onde a latência na detecção de uma ameaça é um fator crítico.

3 Fundamentação Teórica

O processamento de grandes volumes de dados, ou *Big Data*, é frequentemente caracterizado pelos “cinco Vs”: volume, variedade, velocidade, veracidade e valor. Em cenários onde dados fluem continuamente de fontes heterogêneas, os sistemas tradicionais de banco de dados enfrentam dificuldades para equilibrar a ingestão de dados em alta velocidade com a necessidade de consultas complexas e precisas.

Para endereçar esse desafio, [MARZ; WARREN \(2015\)](#) propôs a Arquitetura Lambda, um paradigma de processamento de dados projetado para ser tolerante a falhas e linearmente escalável. A premissa central dessa arquitetura é decompor o problema de computação em três camadas distintas e independentes, que são descritas a seguir:

3.1 Arquitetura Lambda

Batch Layer: É responsável por gerenciar o “*master dataset*” (conjunto de dados mestre), que é imutável e contém todos os dados brutos já ingeridos pelo sistema. Esta camada pré-calcula visualizações em lote (*batch views*), garantindo precisão e consistência, embora com alta latência. A imutabilidade aqui é chave: em caso de erros lógicos no código, os dados brutos permanecem intactos, permitindo o reprocessamento e a correção das visualizações sem perda de integridade.

Speed Layer: Atua para compensar a alta latência da *Batch Layer*. Ela processa os fluxos de dados recentes em tempo real, fornecendo visualizações de baixa latência. Diferente da *Batch Layer*, as visualizações aqui são temporárias e descartadas assim que os dados correspondentes são processados e consolidados pela *Batch Layer*.

Serving Layer: É responsável por indexar e expor as visualizações geradas tanto pela *Batch Layer* quanto pela de *Speed Layer*, permitindo que aplicações clientes consultem os dados de forma unificada e transparente.

Formalmente, essa estratégia pode ser descrita matematicamente. Considerando D como o conjunto total de dados, uma consulta ideal seria $Q = f(D)$. Na Arquitetura Lambda, D é dividido em dados históricos M (*Master Dataset*) e dados recentes R . Assim, a consulta final é obtida pela fusão ($\oplus$) das visualizações da *Batch Layer* (V_b) e da *Speed Layer* (V_s):

$$Q = V_b \oplus V_s = f(M) \oplus f(R) \quad (3.1)$$

Essa abordagem híbrida permite que o sistema tire proveito da eficiência do processamento *offline* (para grandes volumes históricos) e da agilidade do processamento *online* (para

eventos recentes), sendo ideal para ambientes que exigem robustez e atualização constante.

A robustez e a tolerância a falhas são características intrínsecas exigidas para ambientes voláteis e heterogêneos, como os encontrados na Internet das Coisas (IoT). [Veith et al. \(2016\)](#) argumentam que a Arquitetura Lambda oferece as propriedades chave para lidar com essa complexidade, tais como escalabilidade, generalização e leituras de baixa latência, permitindo estratégias de agendamento que equilibram o uso de recursos computacionais. Contudo, é importante notar que a manutenção de duas bases de código distintas (para as camadas *batch* e *speed*) pode introduzir complexidade operacional e custos de desenvolvimento, embora forneça uma garantia de consistência eventual e recuperação de falhas.

3.2 Análise e Seleção de Ferramentas

Para materializar a Arquitetura Lambda proposta, a seleção tecnológica priorizou ferramentas do ecossistema *open-source* que oferecem escalabilidade horizontal, tolerância a falhas e suporte nativo a processamento distribuído. A Figura 1 ilustra o fluxo de dados projetado, desde a ingestão até a *Serving Layer*.

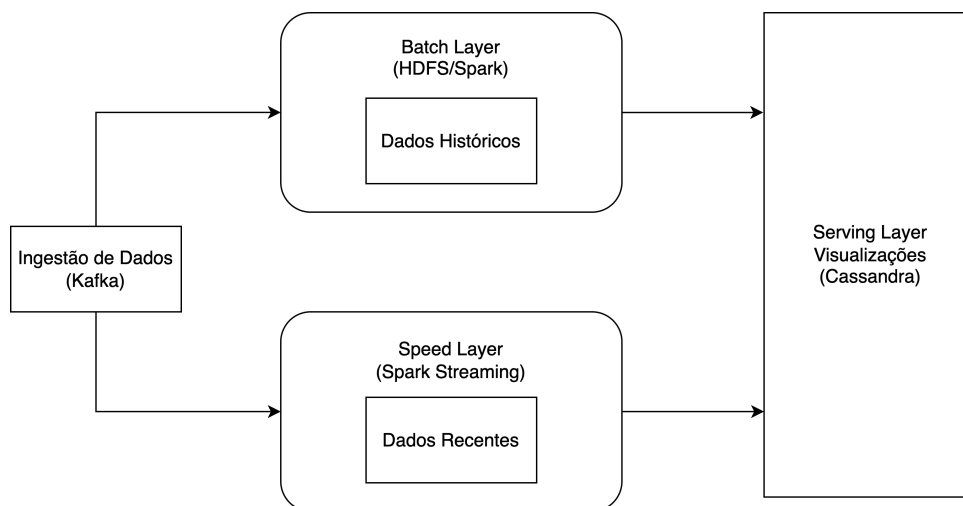


Figura 1 – Fluxo de dados projetado

Os capítulos a seguir apresentam a justificativa para a seleção de cada componente tecnológico, relacionando suas capacidades técnicas aos requisitos essenciais da Arquitetura Lambda. O Capítulo 3.2.1 aprofunda o papel do Apache Kafka, enquanto o Capítulo 3.2.2 descreve em detalhe a utilização do Apache Spark e do HDFS tanto na *Speed Layer* quanto na *Batch Layer*. Por fim, o Capítulo 3.2.3 discute a *Serving Layer*, enfatizando o uso do Apache Cassandra.

3.2.1 Apache Kafka

Como ponto de entrada para as camadas de *Batch* e *Speed*, selecionou-se o Apache Kafka. Sua escolha justifica-se pela capacidade de atuar como um *buffer* de alta vazão e baixa latência. Fundamental para a Arquitetura Lambda, o Kafka garante a persistência das mensagens por um período configurável, permitindo que a *Batch Layer* consuma dados históricos (reprocessamento) enquanto a *Speed Layer* consome os mesmos dados em tempo real, desacoplando os produtores de dados dos consumidores.

A utilização do Apache Kafka é corroborada por [Lekkala \(2020\)](#), que o identificam como ideal para carregar grandes volumes de dados em tempo real na Arquitetura Lambda devido à sua tolerância a falhas e alta vazão. Além disso, sua capacidade de atuar como mediador permite que geradores de dados transmitam fluxos (*broadcast streams*) de forma confiável para consumidores na *Speed Layer*, mitigando gargalos de ingestão.

3.2.2 Processamento e Armazenamento: Apache Spark e HDFS

Para o processamento, optou-se pelo Apache Spark como motor unificado. Diferente de implementações tradicionais da Lambda que utilizam tecnologias distintas para cada camada (como Hadoop MapReduce para *Batch* e Apache Storm para *Speed*), o uso do Spark simplifica a arquitetura ao permitir o compartilhamento de lógica de negócios:

- ***Batch Layer***: Utiliza o Spark Core para processar grandes volumes de dados históricos armazenados no **Hadoop Distributed File System (HDFS)**. O HDFS foi escolhido para hospedar o “*Master Dataset*” devido à sua robustez e natureza *write-once-read-many*, garantindo a imutabilidade exigida pela teoria de [Marz e Warren \(2015\)](#).
- ***Speed Layer***: Emprega o Spark Streaming, que processa os fluxos de dados vindos do Kafka em micro-lotes (*micro-batches*), provendo latência na ordem de segundos, adequada para os requisitos deste trabalho.

3.2.3 *Serving Layer* – Apache Cassandra

Para a *Serving Layer*, escolheu-se o Apache Cassandra, um banco de dados NoSQL orientado a colunas largas (*wide-column store*). A escolha deve-se à sua alta disponibilidade e escalabilidade linear de escrita. O Cassandra é ideal para receber as atualizações frequentes da *Speed Layer* e as substituições em lote da *Batch Layer*, permitindo leituras rápidas para as aplicações finais. Sua arquitetura descentralizada elimina pontos únicos de falha, alinhando-se à robustez geral do sistema proposto.

Ao consolidar a arquitetura proposta, a seleção dessas ferramentas demonstra um alinhamento direto entre os requisitos fundamentais da Arquitetura Lambda — escalabilidade,

tolerância a falhas, imutabilidade e processamento unificado — e as capacidades técnicas oferecidas por cada tecnologia. O Apache Kafka garante ingestão contínua e desacoplamento eficiente; o Apache Spark, atuando nas camadas de *Batch* e *Speed*, reduz a complexidade ao unificar o processamento; o HDFS mantém a integridade do *Dataset*; e o Apache Cassandra fornece consultas rápidas e alta disponibilidade na *Serving Layer*. Assim, a combinação dessas ferramentas não apenas atende às demandas teóricas da Lambda, mas também oferece um ecossistema coeso, maduro e amplamente validado em cenários de produção, justificando plenamente sua adoção.

4 Metodologia

A metodologia deste trabalho fundamenta-se na implementação completa de uma arquitetura Lambda moderna, seguindo os princípios amplamente descritos na literatura por Marz e Warren ([MARZ; WARREN, 2015](#)), bem como em aplicações reais em sistemas distribuídos de alta escala. Essa arquitetura foi escolhida por permitir simultaneamente baixa latência, tolerância a falhas, escalabilidade horizontal e precisão de longo prazo, conciliando processamento em lote e processamento em fluxo.

Além disso, conceitos fundamentais sobre as diferenças entre processamento *batch* e *stream* foram incorporados a partir da análise técnica apresentada em ([BENJELLOUN et al., 2020](#)).

4.1 Cenário de Validação: Dados do SUS

Para validar a arquitetura proposta em um cenário real de *Big Data*, serão utilizados dados públicos do Sistema Único de Saúde (SUS), disponibilizados pelo DATASUS. Este domínio foi escolhido por apresentar as características típicas de volume, velocidade e variedade: o fluxo contínuo de notificações de agravos e internações exige monitoramento em tempo real (para detecção de surtos), enquanto a consolidação de estatísticas epidemiológicas anuais demanda processamento em lote rigoroso.

4.2 Configuração e Orquestração da Infraestrutura

Toda a infraestrutura será provisionada utilizando Docker, garantindo reprodutibilidade e isolamento do ambiente. O uso de *clusters* distribuídos com Kafka, Spark, HDFS e Cassandra segue as recomendações observadas em arquiteturas Lambda aplicadas a domínios críticos, como plataformas orientadas a processamento em tempo real ([ALGHAMDI; BELLAICHE, 2023](#)).

O ecossistema configurado incluirá:

- um *cluster* Kafka para ingestão distribuída de dados;
- um *cluster* Spark (Master + Workers) para processamento paralelo;
- um HDFS (NameNode + DataNodes) para armazenamento imutável da *Batch Layer*;
- um *cluster* Cassandra para disponibilização de visões consolidadas.

Essa combinação reflete as boas práticas discutidas na literatura para ecossistemas de dados em larga escala.

4.2.1 Configuração do Kafka

O Kafka será configurado com múltiplas partições por tópico, permitindo paralelismo no consumo. A política de *acknowledgment* será definida como `all` (`acks=all`) para garantir durabilidade, seguindo as recomendações gerais de sistemas orientados a fluxos contínuos (MARZ; WARREN, 2015).

4.2.2 Configuração do HDFS

O HDFS será configurado com blocos de 128 MB e fator de replicação 2, refletindo os requisitos de tolerância a falhas presentes em arquiteturas de *Big Data* distribuídas. Verificações de integridade periódicas serão executadas com `fsck`, permitindo identificar blocos corrompidos ou replicados.

4.3 Implementação da *Speed Layer*

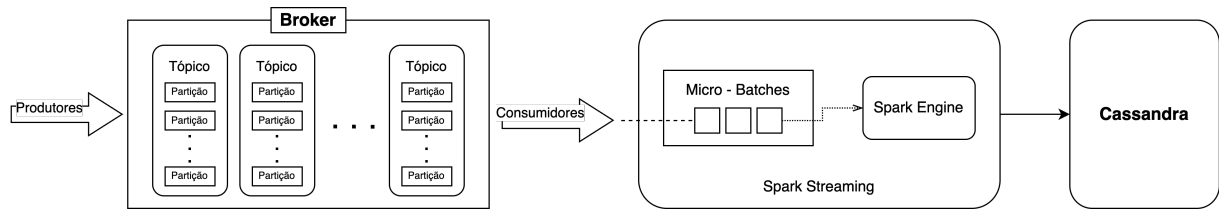
A *Speed Layer*, conforme definida na arquitetura Lambda (MARZ; WARREN, 2015), será implementada com Spark Structured Streaming, processando *micro-batches* de 1 segundo. Esse mecanismo é semelhante ao sistema descrito em (MALUSARE, 2019), que detalha técnicas de integração Spark–Kafka.

O *pipeline* da *Speed Layer* (Figura 2) inclui:

- leitura contínua das partições Kafka;
- normalização e validação dos dados recebidos, para garantir qualidade e consistência, além de evitar erros no *pipeline*;
- agregações em janelas móveis curtas, para produzir resultados em *near-realtime*;
- escrita incremental em Cassandra.

Esse padrão é coerente com o papel da *Speed Layer* em aplicações de baixa latência.

Uma abordagem relevante para otimizar a *Speed Layer* é a mitigação do problema de inicialização a frio (*cold-start*). Pal, Li e Atkinson (2018) propõem um modelo onde a camada de fluxo pode interagir com dados históricos da *Batch Layer* para inicializar o processamento com um *offset* de conhecimento prévio, permitindo inferências mais precisas desde o início da ingestão.

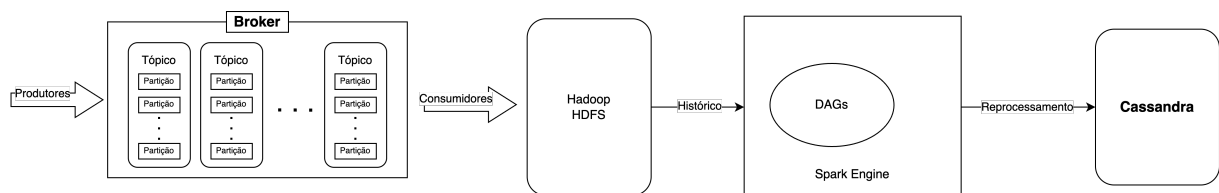
Figura 2 – Pipeline *Speed Layer*.

4.4 Implementação da *Batch Layer*

A *Batch Layer* executa reprocessamentos completos sobre o histórico armazenado no HDFS. Isso garante alta precisão e capacidade de corrigir inconsistências acumuladas na *Speed Layer*.

O pipeline da *Batch Layer* (Figura 3) compreende:

1. leitura completa dos diretórios HDFS, para reprocessar o histórico completo;
2. execução de transformações distribuídas em DAGs Spark, garantindo alta velocidade devido ao paralelismo, além de tolerância a falhas;
3. cálculo preciso dos dados históricos, para corrigir erros ou atrasos do *streaming*;
4. escrita das visões consolidadas em Cassandra.

Figura 3 – Pipeline da *Batch Layer*.

4.5 *Serving Layer*

A *Serving Layer* fornece uma visão unificada das bases *speed* e *batch*, como descrito originalmente em (MARZ; WARREN, 2015). A seleção entre visão rápida ou histórica depende do *timestamp* solicitado.

A Tabela 1 apresenta uma comparação entre a *Speed Layer* e a *Batch Layer*, destacando as propriedades de cada um.

Tabela 1 – Comparação entre *Batch* e *Speed Layer*.

Critério	<i>Speed Layer</i>	<i>Batch Layer</i>
Latência	Baixa	Alta
Precisão	Aproximada	Exata
Persistência	Temporária	Permanente
Papel	Atualização em tempo real	Reprocessamento total

O Cassandra foi escolhido como tecnologia para a *Serving Layer* devido à sua arquitetura distribuída orientada à alta disponibilidade — mesmo em cenários de falha de nós, o *cluster* permanece operacional graças ao modelo *masterless* e à replicação configurável. Além disso, sua escalabilidade horizontal praticamente linear permite lidar com aumentos no volume de dados e nas requisições de leitura sem reestruturações complexas. Outro fator determinante é sua baixa latência tanto para leituras quanto para escritas, característica essencial para servir *views* pré-computadas geradas pelo *Batch Layer* e atualizadas em tempo quase real pelo *Speed Layer*.

4.6 Monitoramento e Observabilidade

Monitoramento é indispensável em sistemas distribuídos. Painéis de visualização serão implementados em Grafana, consumindo métricas expostas via JMX e coletadas pelo Prometheus.

A Figura 4 apresenta a arquitetura de monitoramento do sistema, na qual os componentes distribuídos — como Kafka, Spark e Hadoop — expõem métricas operacionais por meio do protocolo JMX. Essas métricas são coletadas pelo Prometheus, que atua como responsável pela extração, armazenamento e organização das séries temporais. Em seguida, o Grafana consome esses dados para a construção de *dashboards* que permitem acompanhar, em tempo real, o comportamento e o desempenho do *pipeline*, proporcionando maior observabilidade e facilitando a detecção de falhas e gargalos.

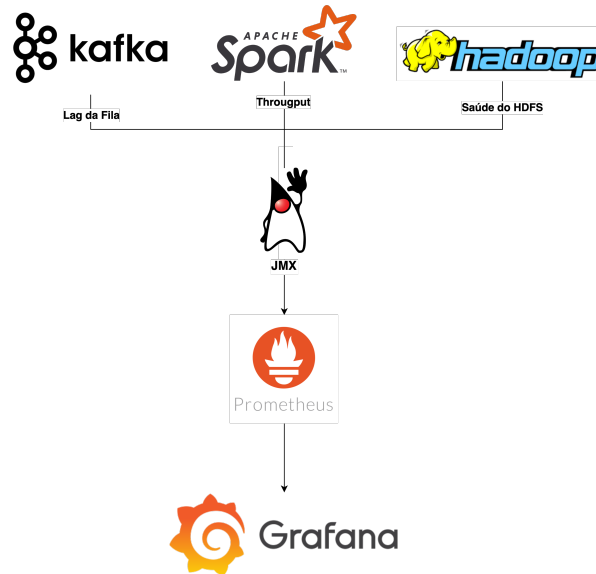


Figura 4 – Monitoramento do sistema.

4.7 Testes, Validação e Robustez

A resiliência da arquitetura será verificada por meio de três tipos de testes, cada um selecionado para avaliar um aspecto crítico do funcionamento de sistemas distribuídos:

- **queda de partição Kafka:** este teste simula a indisponibilidade de uma partição no Kafka, componente responsável pela ingestão de dados em tempo real. A escolha desse cenário se deve ao fato de que falhas em partições podem gerar atrasos, perda temporária de acesso a mensagens ou aumento no *lag* de consumo. Avaliar esse caso permite verificar se o sistema consegue retomar o processamento sem perda de dados e medir quanto tempo leva para estabilizar novamente;
- **queda de Spark Worker:** nesse teste, um nó do *cluster* Spark é removido deliberadamente, simulando falhas de *hardware* ou interrupções inesperadas. Esse cenário foi escolhido porque o Spark, presente na *Speed Layer*, deve ser capaz de redistribuir as tarefas automaticamente entre os demais *workers*, mantendo a continuidade do processamento em tempo real. Medir a recuperação desse componente é essencial para garantir que o *pipeline* sobreviva a falhas de execução;
- **reprocessamento de longo período na Batch Layer:** aqui, força-se o reprocessamento de um volume histórico significativo (por exemplo, 30 dias de dados). O objetivo desse teste é confirmar que o sistema consegue reconstruir a *Batch View* de forma íntegra e, ao final, garantir consistência entre os resultados produzidos pelo *Batch Layer* e pelo *Speed Layer*. A escolha se justifica por ser um dos cenários mais importantes para assegurar precisão e reprodutibilidade dos dados.

Além disso, usaremos o Tempo Médio de Recuperação (MTTR) para calcular o tempo médio que o sistema leva para se recuperar de uma falha e retornar ao funcionamento normal. Ele é amplamente utilizado para avaliar a resiliência de sistemas distribuídos.

O MTTR será formalmente calculado conforme a Equação 4.1:

$$MTTR = \frac{\sum t_{down}}{N_f} \quad (4.1)$$

Onde t_{down} representa o tempo total de indisponibilidade durante os testes e N_f o número total de falhas induzidas.

A Tabela 2 mostra um resumo dos testes realizados e seus resultados esperados.

Tabela 2 – Resumo dos testes de robustez.

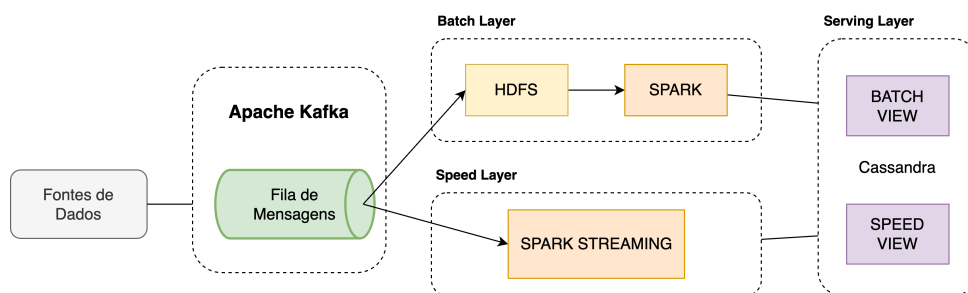
Teste	Descrição	Resultado Esperado
Falha Kafka	Interrupção de partição	Recuperação sem perda de dados e medição do MTTR
Falha Spark Worker	Remoção de nó	Redistribuição automática e medição do MTTR
Reprocessamento <i>Batch</i>	Reprocessamento de 30 dias de dados	Consistência final entre visões <i>batch</i> e <i>speed</i>

A validação experimental da arquitetura deve ir além da recuperação de falhas, focando também na precisão do processamento de dados. Demirezen e Navruz (2023) demonstram, através de estudos empíricos em sistemas de vigilância aérea, que a Arquitetura Lambda é “eventualmente precisa” (*eventually accurate*). Seus experimentos indicam que, apesar de possíveis inconsistências transitórias na *Speed Layer*, o sistema converge para resultados confiáveis após o processamento da *Batch Layer*, validando a eficácia do modelo em cenários críticos onde a precisão dos dados é fundamental.

4.8 Arquitetura Consolidada

A Figura 5 mostra como a arquitetura final foi consolidada. Ela mostra desde a entrada dos dados, passando pela fila Kafka, a entrada para as camadas *batch* e *speed* até, finalmente, chegar na *Serving Layer*.

Figura 5 – Arquitetura final do sistema.



Fonte: o Autor, adaptado de (MALUSARE, 2019).

5 Plano de Trabalho e Cronograma

O plano de atividades deste projeto envolve diversas etapas, desde a revisão bibliográfica inicial até a escrita da monografia final (TCC2). O cronograma abaixo contempla as atividades realizadas no TCC I, bem como aquelas previstas para o TCC II. As atividades foram organizadas de forma a garantir um progresso contínuo e focando na conclusão bem-sucedida do trabalho.

Atividades	Mês 1	Mês 2	Mês 3	Mês 4	Mês 5	Mês 6	Mês 7	Mês 8	Mês 9	Mês 10
Definição do problema (TCC1)	✓									
Revisão bibliográfica (TCC1)		✓	✓							
Elaboração do plano de pesquisa (TCC1)			✓							
Refinamento do projeto (TCC1)			✓	✓						
Análise das ferramentas (TCC1)			✓	✓						
Escrita da monografia (TCC1)			✓	✓	✓					
Melhoria bibliográfica (TCC2)						✓				
Construção do ambiente (Infra) (TCC2)						✓	✓	✓		
Criação das massas de teste (TCC2)							✓	✓		
Avaliação da arquitetura (TCC2)								✓	✓	
Escrita da monografia final (TCC2)								✓	✓	✓
Escrita de artigo científico (TCC2)										✓

Tabela 3 – Cronograma de atividades para TCC I e TCC II

Referências

AISSI, M. E. M. E. et al. A scalable smart farming big data platform for real-time and batch processing based on lambda architecture. *Journal of System and Management Sciences*, v. 13, n. 2, p. 17–30, 2023. ISSN 1816-6075 (Print), 1818-0523 (Online). Citado 2 vezes nas páginas [21](#) e [26](#).

ALGHAMDI, R.; BELLAICHE, M. An ensemble deep learning based ids for iot using lambda architecture. *Cybersecurity*, v. 6, n. 5, 2023. Open Access article under Creative Commons Attribution 4.0 International License. Citado 3 vezes nas páginas [22](#), [26](#) e [31](#).

ANDE, K.; SINGH, R. K. The modern data engineer: A comprehensive guide to etl, mdm, and cloud data solutions. *International Journal for Research Publication and Seminar*, v. 16, n. 1, p. 344–362, 2025. Citado na página [22](#).

BENJELLOUN, S. et al. Big data processing: Batch-based processing and stream-based processing. In: *Proceedings of the International Conference on Computing Sciences (ICoCS)*. [S.l.: s.n.], 2020. 978-1-7281-8084-7/20/\$31.00 ©2020 IEEE. Citado 2 vezes nas páginas [25](#) e [31](#).

DEMIREZEN, M. U.; NAVRUZ, T. S. Performance analysis of lambda architecture-based big-data systems on air/ground surveillance application with ads-b data. *Sensors*, MDPI, v. 23, n. 17, p. 7580, 2023. Citado na página [36](#).

LEKKALA, C. Leveraging lambda architecture for efficient real-time big data analytics. *European Journal of Advances in Engineering and Technology*, v. 7, n. 2, p. 59–64, 2020. Disponível em: <www.ejaet.com>. Citado na página [29](#).

MALUSARE, O. A. *Real-Time Data Processing With Lambda Architecture*. Dissertação (Master's Project) — San Jose State University, 2019. Part of the Systems Architecture Commons. Citado 4 vezes nas páginas [21](#), [25](#), [32](#) e [37](#).

MARZ, N.; WARREN, J. *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Shelter Island: Manning Publications, 2015. Licensed to Mark Watson <nordickan@gmail.com>. Citado 8 vezes nas páginas [21](#), [22](#), [25](#), [27](#), [29](#), [31](#), [32](#) e [33](#).

MUNSHI, A. A.; MOHAMED, Y. A.-R. I. Data lake lambda architecture for smart grids big data analytics. *IEEE Access*, v. 6, p. 40463–40471, 2018. Received May 14, 2018, accepted July 15, 2018, date of publication July 23, 2018. Citado 2 vezes nas páginas [21](#) e [26](#).

PAL, G.; LI, G.; ATKINSON, K. Multi-agent big-data lambda architecture model for e-commerce analytics. *Data*, MDPI, v. 3, n. 4, p. 58, 2018. Citado na página [32](#).

QU, J.; WANG, J. Real-time data warehousing in the big data environment: A comprehensive review of implementation in the internet industry. In: *Proceedings of the 6th International Conference on Computing and Data Science*. [S.l.: s.n.], 2024. p. 110–119. Citado na página [25](#).

ROUKH, A. et al. Big data processing architecture for smart farming. *Procedia Computer Science*, v. 177, p. 78–85, 2020. The 11th International Conference on Emerging Ubiquitous

Systems and Pervasive Networks (EUSPN 2020), Open Access under CC BY-NC-ND license. Citado 2 vezes nas páginas [21](#) e [25](#).

VEITH, A. d. S. et al. Strategies for big data analytics through lambda architectures in volatile environments. In: *IFAC-PapersOnLine*. [S.l.]: Elsevier, 2016. v. 49, n. 30, p. 114–119. Citado na página [28](#).